

CS 2110 Final Exam: C

Max Everest, Sebastian Jankowski, Jonathan Marto, Annelise Lloyd, Zaid Akkawi

Summer 2023

Contents

1	Rules - Please Read	2
2	Overview	2
2.1	Description	2
3	Instructions	2
3.1	Useful Structs and Unions	3
3.2	makeWrapper()	3
3.3	replacePearVariety()	3
3.4	Diagram	4
4	Grading	5
5	Deliverables	5
6	Autograder and Debugging	6
6.1	Makefile	6
6.2	Debugging with GDB - List of Commands	7
6.3	Autograder	7
6.4	Valgrind Errors	8

Please take the time to read the entire document before starting the assignment. It is your responsibility to follow the instructions and rules.

1 Rules - Please Read

You are allowed to submit this portion of the final exam starting from the moment your timed lab portion of the exam period begins until your individual period ends. You have 75 minutes to complete *both of the timed lab (C and LC3-Assembly)* portions of the exam, unless you have accommodations that have already been discussed with your professor. Gradescope submissions will close precisely at the end of your allotted exam period. *Note:* The Assembly and C coding sections are independent of each other so you may complete these in any order, therefore there is no enforced time limit to complete either section. Just be mindful of the time you have left!

If you have questions during the exam, you may ask the TAs for clarification, though you are ultimately responsible for what you submit. The information provided in this document takes precedence. If you notice any conflicting information, please indicate it to your TAs.

The timed lab sections of the final exam are open-resource. You may reference your previous homeworks, class notes, etc., but your work must be your own. Contact in any form with any other person besides a TA is absolutely forbidden. **No collaboration is allowed.**

2 Overview

2.1 Description

You have been given two C files - `food.c` and `food.h`. For `food.c`, there are two functions that you need to complete according to the comments and descriptions given below.

3 Instructions

For this section of the final exam, you will help **Kirby track a list of foods that he can consume during a fight**. He consumes pears and potatoes, where he can spit pears out to deal damage to an enemy, and eat potatoes to heal him. You will be writing functions that manage this list of foods. At this point, it would be recommended to open both `food.c` and `food.h` to preview the files, and better understand the following sections.

Remember: You should **not** modify any other files. Doing so may result in point deductions. You should also **not** modify the `#include` statements nor add any more. You are also not allowed to add any global variables. You may however add macros and helper functions as long as they are written in `food.c` and pass the autograder. You will not be submitting `food.h`, so any modifications in `food.h` will not be reflected in the Gradescope autograder.

3.1 Useful Structs and Unions

A struct `wrapper` contains a struct `data`, a struct describing the type of food; struct `wrapper *next`, a pointer to the next struct wrapper in the food queue.

A struct `data` contains a `foodType`, an instance of the `foodType` enum (either `POTATO` or `PEAR`); a union `food`, a union that will hold either a struct `pear` or struct `potato` depending on the food type.

A union `food` contains either a struct `pear`, a struct describing pear properties, **OR** a struct `potato`, a struct describing potato properties.

A struct `pear` contains `int damage`, the damage value this pear deals to enemies; `char *pearVariety`, the string holding the pear variety (ex. Anjou, Bartlett, etc.).

A struct `potato` contains `int health`, the health value this potato gives to Kirby.

3.2 `makeWrapper()`

```
int makeWrapper(foodType foodType, int damage, const char *pearVariety, int health);
```

This function takes in a `foodType` for the type of food contained in the wrapper, an `int` for the amount of damage done if the food in the wrapper is a pear, a pointer to the pear variety if the food in the wrapper is a pear, and an `int` for the amount of health restored if the food in the wrapper is a potato.

The function will create a new wrapper struct that holds an item of food with the given information by allocating memory for it on the heap. After adding the given arguments to the newly created wrapper struct, add this wrapper struct to the front of the food list, given by the head global variable. If this function succeeds, it should return `SUCCESS` and add the newly created wrapper to the front of the list. If it fails, it should return `FAILURE` and leave the list unchanged.

Return `FAILURE` if the `pearVariety` is `NULL` or dynamic memory allocation fails. Otherwise, you will dynamically allocate space for the wrapper, and return `SUCCESS` if you are able to create a new wrapper successfully and add it to the front of the list.

3.3 `replacePearVariety()`

```
replacePearVariety(const char *replacementVariety, const char *targetVariety);
```

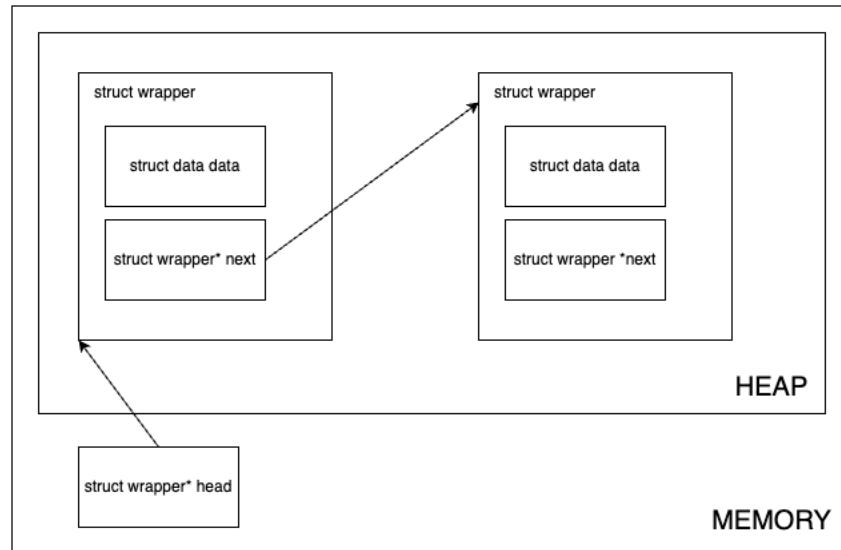
This function takes in a string containing the new `pearVariety`, and a string containing the old `pearVariety` to find in the list.

This function will iterate through the list of food and find the first wrapper with a pear that has a `pearVariety` that is the same as the `targetVariety` in it. For that pear, you will replace its `pearVariety` with the `replacementVariety`.

If any of the following are true, return `FAILURE`: `replacementVariety` is null, `targetVariety` is `NULL`, dynamic memory allocation fails, or `targetVariety` is not found in the list. Otherwise, upon finding the first wrapper containing `pearVariety` that matches `targetVariety`, you will replace `targetVariety` with the `replacementVariety`. Return `SUCCESS` if this replacement was successful.

3.4 Diagram

Refer to the following for a visual representation of structs and unions used in the functions you are meant to implement. In the first diagram, we see how generally multiple struct wrappers are stored on the heap.

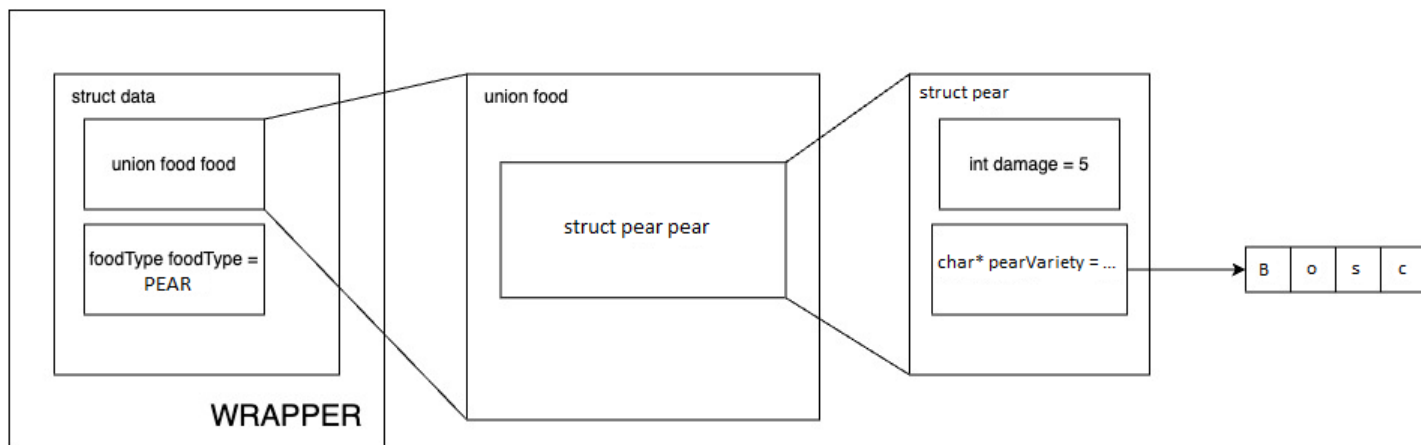


In the second diagram, we see resulting structs and unions when we `makeWrapper(PEAR, 5, "Gala", 7)`.

Within the new wrapper, there is the struct data holding a food union and `enum foodType = PEAR`.

The union food holds a pear struct; note that it does not hold the potato struct, since our `foodType` is `PEAR` for this wrapper instance.

The struct pear holds damage, which is 5, and the string that holds the `pearVariety`.



4 Grading

Point distribution for this portion of the final exam is broken down as follows:

- `makeWrapper`:
 - If the passed in `pearVariety` is `NULL` if the type of food is an `PEAR`, you should return `FAILURE`.
 - If dynamic memory allocation fails at any point, you should return `FAILURE`.
 - If dynamic memory allocation does not fail, then the newly allocated wrapper should be added to the head of the food queue
 - All values passed into the function should be deep copied into the newly created wrapper struct.
 - If the function is successful, you should return `SUCCESS`.
- `replacePearVariety`:
 - If the passed in `replacementVariety` is `NULL`, or the passed in `targetVariety` is `NULL`, you should return `FAILURE`.
 - If the `targetVariety` is not found in the food queue, you should return `FAILURE`.
 - If dynamic memory allocation fails at any point, you should return `FAILURE`.
 - If there is space to replace the `pearVariety` in a wrapper struct, then the `pearVariety` should be set to the passed in `replacementVariety` if the current `pearVariety` is the same as `targetVariety`.
 - If the function is successful, you should return `SUCCESS`.

5 Deliverables

Turn in the following files on Gradescope during your assigned final exam period.

1. `food.c`

Your file must compile with our Makefile, which means it must compile with the following gcc flags:

`-std=c99 -pedantic -Wall -Werror -Wextra -Wstrict-prototypes -Wold-style-definition`

All non-compiling final exam submissions will receive a zero. If you want to avoid this, do not run gcc manually; use the Makefile as described below.

6 Autograder and Debugging

6.1 Makefile

We have provided a Makefile for this final exam section that will build your project. Here are the commands you should be using with this Makefile:

1. To clean your working directory (use this command instead of manually deleting the .o files): `make clean`
2. To compile the code in `main.c`: `make food`
3. To compile the tests: `make tests`
4. To run all tests at once: `make run-tests`
 - To run a specific test: `make run-tests TEST=test_name`
5. To run all tests at once with Valgrind enabled: `make run-valgrind`
 - To run a specific test with Valgrind enabled: `make run-valgrind TEST=test_name`
6. To debug a specific test using gdb: `make run-gdb TEST=test_name`

Then, at the (gdb) prompt:

- (a) Set some breakpoints (if you need to—for stepping through your code you would, but you wouldn't if you just want to see where your code is segfaulting) with `b suites/food_suite.c:43`, or `b food.c:39`, or wherever you want to set a breakpoint
- (b) Run the test with `run`
- (c) If you set breakpoints: you can step line-by-line (including into function calls) with `s` or step over function calls with `n`
- (d) If your code segfaults, you can run `bt` to see a stack trace

To get an individual test name, you can look at the output produced by the tester. For example, the following failed test is `test_change_pear_variety`:

```
suites/food_suite.c:45:F:test_change_pear_Variety:test_change_pear_variety:0:
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Beware that segfaulting tests will show the line number of the last test assertion made before the segfault, not the segfaulting line number itself. This is a limitation of the testing library we use. To see what line in your code (or in the tests) is segfaulting, follow the “To debug a specific test using gdb” instructions above.

6.2 Debugging with GDB - List of Commands

Debug a specific test:

```
$ make run-gdb TEST=test_name
```

Basic Commands:

- `b <function>` **break point** at a specific function
- `b <file>:<line>` **break point** at a specific line number in a file
- `r` **run** your code (be sure to set a break point first)
- `n` **step over** code
- `s` **step into** code
- `p <variable>` **print** variable in current scope (use `p/x` for hexadecimal)
- `bt` **back trace** displays the stack trace (useful for segfaults)

6.3 Autograder

We have provided you with a test suite to check your work. You can run these using the Makefile.

Note: There is a file called `test_utils.o` that contains some functions that the test suite needs. We are not providing you the source code for this, so make sure not to accidentally delete this file as you will need to redownload the assignment. This file is not compiled with debugging symbols, so you will not be able to step into it with `gdb` (which will be discussed shortly).

We recommend that you write the function according to the pdf and file comments to ensure that you are meeting all the requirements before moving onto testing and debugging. Then, you can make sure that you do not have any memory leaks using Valgrind. It doesn't pay to run Valgrind on tests that you haven't passed yet. Below, there are instructions for running Valgrind on an individual test under the Makefile section, as well as how to run it on all of your tests.

The given test cases are the same as the ones on Gradescope. We formally reserve the right to change test cases or weighting after the lab period is over. However, if you pass all the tests and have no memory leaks according to Valgrind, you can be confident that you will get 100% as long as you did not cheat or hard code in values.

Printing out the contents of your structures can't catch all logical and memory errors, which is why we also require you run your code through Valgrind. You will not receive credit for any tests you pass where Valgrind detects memory leaks or memory errors. Gradescope will run Valgrind on your submission, but you may also run the tester locally with Valgrind for ease of use.

We certainly will be checking for memory leaks by using Valgrind, so if you learn how to use it, you'll catch any memory errors before we do.

Your code must not crash, run infinitely, nor generate memory leaks/errors.

Any test we run for which Valgrind reports a memory leak or memory error will receive no credit.

If you need help with debugging, there is a C debugger called `gdb` that will help point out problems. See instructions in the Makefile section for running an individual test with `gdb`.

6.4 Valgrind Errors

If you mishandle memory in C, chances are you will lose half or all of a test's credit due to a Valgrind error. You can find a comprehensive guide to Valgrind errors here: <https://valgrind.org/docs/manual/mc-manual.html#mc-manual.errormsgs>

For your convenience, here is a list of common Valgrind errors:

- **Illegal read/write:** this happens when you read or write to memory that was not allocated using malloc/calloc/realloc. This can happen if you write to memory that is outside a buffer's bounds, or if you try to use a recently freed pointer. If you have an illegal read/write of 1 byte, then there is likely a string involved; you should make sure that you allocated enough space for all your strings, including the null terminator.
- **Conditional jump or move depends on uninitialized value:** this usually happens if you use malloc or realloc to allocate memory and forget to initialize the memory. Since malloc and realloc do not manually clear out memory, you cannot assume that it is full of zeros.
- **Invalid free:** this happens if you free a pointer twice or try to free something that is not heap-allocated. Usually, you won't actually see this error, since it will often cause the program to halt with an Signal 6 Aborted message.
- **Memory leak:** this happens if you forget to free something. The memory leak printout should tell you the location where the leaked data is allocated, so that hopefully gives you an idea of where it was created. Remember that you must free memory if it is not being returned from a function. (Think about what you had to do for empty_list in HW9!)